College CS DSA Class Guide

Audience: undergraduate computer science students taking a Data Structures and Algorithms course.

1. Course Goal

By the end of the class, a student should be able to:

- Explain common data structures and algorithms.
- Analyze time and space complexity.
- Implement core algorithms correctly.
- Choose the right structure for a problem.
- Solve interview-style and academic DSA problems.
- Communicate tradeoffs clearly.

2. How to Study DSA

1. **Understand the concept.** Learn what the structure or algorithm does.
2. **Trace by hand.** Use small examples and draw memory/state changes.
3. **Implement from scratch.** Do not memorize only library calls.
4. **Analyze complexity.** Identify input size and dominant operations.
5. **Solve patterns.** Practice variations until the pattern is recognizable.
6. **Review mistakes.** For every wrong answer, write why it failed.

3. Core Topic Roadmap

Unit	Topics	What to master
1	Complexity Analysis	Big O, loops, recursion, time-space tradeoffs
2	Arrays and Strings	Indexing, two pointers, sliding window, prefix sums
3	Linked Lists, Stacks, Queues	Pointers, LIFO, FIFO, cycle detection
4	Recursion	Base cases, call stack, recurrence thinking
5	Sorting and Searching	Merge sort, quicksort, binary search, boundaries
6	Hashing	Hash maps, sets, frequency tables, collisions
7	Trees, Heaps, Tries	Traversals, priority queues, prefix trees
8	Graphs	BFS, DFS, topological sort, shortest paths
9	Greedy and Backtracking	Local choices, pruning, permutations, subsets
10	Divide and Conquer, Dynamic Programming	Recurrences, memoization, tabulation

4. Big O Quick Reference

Complexity	Meaning	Example
O(1)	Constant	Array index access
O(log n)	Logarithmic	Binary search
O(n)	Linear	Single pass through array

O(n log n)	Linearithmic	Merge sort
O(n^2)	Quadratic	Nested loop over pairs
O(2^n)	Exponential	Subset generation
O(n!)	Factorial	Permutation generation

5. Topic Summaries

Arrays

Arrays store values by index. Use them when you need fast random access.

- Access: O(1)
- Search: O(n)
- Insert/delete middle: O(n)
- Common patterns: two pointers, sliding window, prefix sums

Strings

Strings are sequences of characters. Many string problems are array problems with character-specific rules.

- Common patterns: frequency map, palindrome check, substring window, hashing
- Watch for case sensitivity and Unicode/encoding constraints.

Linked Lists

Linked lists store nodes connected by references.

- Access: O(n)
- Insert/delete known node: O(1)
- Common patterns: fast/slow pointers, reversal, dummy head

Stacks

Stacks are last-in-first-out.

- Use for parentheses, undo, recursion simulation, monotonic stack problems.
- Push/pop/peek: O(1)

Queues

Queues are first-in-first-out.

- Use for BFS, task scheduling, producer-consumer systems.
- Enqueue/dequeue: O(1) with the right implementation.

Recursion

Recursion solves a problem by solving smaller versions of itself.

- Always define a base case.
- Track what each call returns.
- Space complexity includes call stack depth.

Sorting

Sorting rearranges data according to an order.

- Merge sort: O(n log n), stable, O(n) extra space.

- Quicksort: average O(n log n), worst O(n^2), often fast in practice.
- Sorting often turns hard search problems into easier two-pointer problems.

Searching

Searching finds a value or boundary.

- Linear search: O(n)
- Binary search: O(log n), requires sorted or monotonic condition
- Be careful with off-by-one errors.

Hashing

Hashing maps keys to values for fast lookup.

- Average insert/search/delete: O(1)
- Space: O(n)
- Use for duplicates, counts, complements, grouping, and memoization.

Trees

Trees model hierarchy.

- DFS uses recursion or stack.
- BFS uses queue.
- Balanced BST search is O(log n); unbalanced tree search can be O(n).

Heaps

Heaps support priority access.

- Peek min/max: O(1)
- Insert/extract: O(log n)
- Use for top-k, scheduling, streaming median, and Dijkstra.

Tries

Tries store strings by prefix.

- Insert/search: O(k), where k is word length.
- Use for autocomplete, dictionary search, and prefix matching.

Graphs

Graphs model relationships.

- BFS/DFS: O(V + E)
- Use adjacency lists for sparse graphs.
- Common problems: components, shortest path, cycle detection, course scheduling.

Dynamic Programming

DP solves overlapping subproblems.

- Identify state.
- Define transition.
- Choose memoization or tabulation.
- Optimize space when only recent states are needed.

Greedy Algorithms

Greedy algorithms choose the best local option each step.

- Works only when local optimal choices lead to global optimum.
- Common in intervals, scheduling, heaps, and some graph problems.

Backtracking

Backtracking explores possible choices and undoes them.

- Use for combinations, permutations, N-Queens, Sudoku.
- Add pruning to reduce search.

Divide and Conquer

Divide a problem, solve subproblems, combine results.

- Examples: merge sort, quicksort, binary search, fast power.
- Analyze with recurrences.

6. Weekly Practice Plan

Practice block	Activity
Concept review	Read notes and redraw examples.
Implementation	Code the structure or algorithm from scratch.
Easy problems	Build confidence with direct applications.
Medium problems	Practice pattern recognition.
Reflection	Record mistakes, complexity, and alternate solutions.

7. Problem-Solving Template

Use this template before coding:

1. Restate the problem in your own words.
2. Identify input size and constraints.
3. Write two small examples.
4. Choose the data structure.
5. Start with brute force.
6. Improve complexity.
7. Write pseudocode.
8. Code carefully.
9. Test edge cases.
10. Explain time and space complexity.

8. Common Interview Questions by Topic

Topic	Practice questions
Arrays	Two Sum, Maximum Subarray, Rotate Array
Strings	Valid Anagram, Longest Substring Without Repeating Characters
Linked Lists	Reverse List, Detect Cycle, Merge Sorted Lists
Stacks	Valid Parentheses, Min Stack, Daily Temperatures
Queues	Number of Islands, Rotting Oranges, Level Order Traversal

Recursion	Generate Parentheses, Permutations
Sorting	Merge Intervals, Sort Colors
Searching	Binary Search, Find Minimum in Rotated Sorted Array
Hashing	Group Anagrams, Subarray Sum Equals K
Trees	Validate BST, Lowest Common Ancestor
Heaps	Top K Frequent Elements, Kth Largest Element
Tries	Implement Trie, Word Search II
Graphs	Course Schedule, Clone Graph
DP	Coin Change, Longest Increasing Subsequence
Greedy	Jump Game, Non-overlapping Intervals
Backtracking	Subsets, Combination Sum, N-Queens
Divide and Conquer	Fast Power, Merge Sort

9. Mini Coding Examples

Binary Search

```
function binarySearch(nums: number[], target: number): number {
  let left = 0;
  let right = nums.length - 1;

  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);
    if (nums[mid] === target) return mid;
    if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
  }

  return -1;
}
```

DFS on a Tree

```
type TreeNode = {
  value: number;
  left: TreeNode | null;
  right: TreeNode | null;
};

function preorder(root: TreeNode | null, out: number[] = []): number[] {
  if (!root) return out;
  out.push(root.value);
  preorder(root.left, out);
  preorder(root.right, out);
```

```
  return out;
}
```

Dynamic Programming: Climbing Stairs

```
function climbStairs(n: number): number {
  let prev = 1;
  let curr = 1;
  for (let step = 2; step <= n; step++) {
    const next = prev + curr;
    prev = curr;
    curr = next;
  }
  return curr;
}
```

10. Exam Preparation Checklist

- I can explain Big O for loops, nested loops, and recursion.
- I can implement arrays, stacks, queues, linked lists, trees, heaps, and tries.
- I can perform BFS and DFS on trees and graphs.
- I can write binary search without off-by-one bugs.
- I know when to use a hash map.
- I can solve at least one problem using greedy reasoning.
- I can solve DP problems using memoization and tabulation.
- I can explain the tradeoff between time and space.
- I can test edge cases before submitting code.